



中南大學
CENTRAL SOUTH UNIVERSITY

微控制器与嵌入式系统 课程设计报告

题目

基于神经网络的
手写数字识别系统

学生姓名

金奕帆

学号

8202240417

专业班级

自 T2401

学院

自动化学院

本科生院制

2026 年 7 月

摘要

本课程设计围绕任务书题目 6“基于神经网络的手写数字识别系统”展开，采用野火 STM32F103VE 指南者开发板作为下位机，完成触摸屏手写轨迹采集、LCD 实时笔迹显示、28 x 28 灰度图像预处理、量化神经网络推理和串口上报。PC 端使用 PyTorch 训练 Perceptron、FNN、Tiny-CNN 等数字识别模型，并将权重导出为 C 数组存放在 STM32 Flash 中；字母扩展部分基于 EMNIST Letters 训练 Letter-Perceptron、Letter-FNN 和 Letter-DS-CNN，形成与数字域互不混烧的独立固件。

系统配套开发了网页上位机 Dashboard，支持浏览器/单片机输入切换、板端实时轨迹显示、模型输入像素化展示、P/F/C 三模型置信度显示、样本一键采集、自动化测试、Keil 构建烧录、中文识别 API 原型和固件资源可视化。进阶部分完成了标准 MNIST、个人采集、USPS 外部数据集和 EMNIST Letters 测试集组织，对准确率、平均推理时间和易混淆类别进行统计。同时设计并联动跨平台传输与边缘协同推理方案：STM32 可将 RESULT 或 IMAGE 帧通过串口发送到华清远见 FS6818M4 Linux 实验箱，由实验箱进行语音播报或部署更大 CNN 完成数字/字母识别。

测试结果表明，在 1000 张 MNIST 标准测试集上，Perceptron、FNN、Tiny-CNN 的准确率分别为 87.40%、92.90%、95.80%；在 129 张个人采集集上，FNN 与 Tiny-CNN 均达到 99.22%。字母识别中 Letter-DS-CNN 在 20800 张 EMNIST Letters 测试集上达到 90.09%。最终 Keil 数字固件 Flash 占用约 284.6 KB、SRAM 占用约 17.6 KB；字母固件 Flash 占用约 361.4 KB、SRAM 占用约 46.8 KB，均满足 STM32F103VE 的 512 KB Flash / 64 KB SRAM 约束。

关键词：STM32F103VE；手写数字识别；MNIST；EMNIST；量化神经网络；网页上位机；边缘协同推理

目录

摘要	2
1 课程设计任务与需求分析	5
1.1 题目背景	5
1.2 任务书要求与完成情况	5
1.3 设计目标	5
2 总体方案设计	6
2.1 系统总体架构	6
2.2 数据流设计	6
2.3 软硬件模块划分	6
3 硬件系统设计	7
3.1 主控与外设选择	7
3.2 接口连接关系	7
3.3 调试与下载方式	7
4 软件系统设计	7
4.1 固件目录与核心文件	7
4.2 串口协议设计	7
4.3 网页上位机设计	8
4.4 构建与烧录流程	8
5 神经网络模型与量化部署	9
5.1 四类模型设计	9
5.2 数字识别模型	10
5.3 字母识别模型	10
5.4 int8 量化方法	10
5.5 中文识别与大字符集方案	10
6 关键算法与程序设计	11

6.1 手写轨迹预处理	11
6.2 定点推理	11
6.3 模型导出与固件同步	11
6.4 代码规范落实	11
7 测试集制作与自动化测试	11
7.1 测试集组织	11
7.2 自动化测试方法	12
7.3 测试结果	12
7.4 易混淆类别分析	13
8 系统联调与资源分析	13
8.1 Keil 编译与下载	13
8.2 Flash/SRAM 占用	13
8.3 实物交互效果	13
9 跨平台传输与边缘协同推理	13
9.1 跨平台传输：STM32 到 FS6818M4 语音播报	13
9.2 边缘协同推理：STM32 采集，Linux 推理	14
9.3 通信接口扩展	14
10 特色创新与不足	14
10.1 特色创新	14
10.2 不足与改进方向	14
11 总结	15
参考文献	15
附录 A 主要文件清单	15
附录 B 常用运行命令	16

1 课程设计任务与需求分析

1.1 题目背景

手写数字识别是模式识别和机器学习中最经典的任务之一，既能体现图像采集、预处理、特征提取、模型训练与部署的完整链路，又适合在资源有限的微控制器平台上验证嵌入式 AI 的可行性。本设计选择 STM32F103VE 开发板作为 MCU 端，利用触摸屏作为手写输入设备，通过 LCD 进行交互显示，并在 PC/Linux 端提供训练、测试和可视化工具。

1.2 任务书要求与完成情况

根据课程设计任务书，基本任务包括触摸屏采集、单层感知机训练、模型权重导出和 STM32 端推理；进阶任务包括 FNN、测试集、自动化测试、跨平台传输、CNN、边缘协同推理和多类型字符识别。表 1-1 给出本项目完成情况。

表 1-1 任务要求与完成情况

类别	任务要求	完成状态	本项目实现
基本任务 1	触摸屏输入采集与 LCD 实时轨迹	已完成	XPT2046 触摸采样、LCD 画板、串口 POINT/STROKE/IMAGE 帧
基本任务 2	MNIST 单层感知机训练并保存权重	已完成	PyTorch 训练，导出 PerceptronData.c/h
基本任务 3	单层感知机部署到 STM32 Flash 并推理	已完成	int8 权重 + int32 累加，LCD/串口输出 P 结果
进阶 1/2	FNN 训练、导出、部署与推理	已完成	FNN_Data.c/h, P/F/C 三模型对比
进阶 3	TF 卡测试集制作	已完成	tf_card/mnist、personal、external_usps、emnist_letters, label.txt/manifest.csv
进阶 4	自动化测试准确率与耗时统计	已完成	Web 自动化测试页与 evaluate_tf_card.py, 统计混淆对
进阶 5	跨平台传输到华清远见实验箱语音播报	已完成方案与接口	STM32 输出 RESULT, Linux 端串口读取并 TTS 播放结果
进阶 6	CNN 模型训练与部署	已完成轻量方案	数字 Tiny-CNN, 字母 DS-CNN; Linux 协同侧可部署更大 CNN
进阶 7	边缘协同推理	已完成方案与接口	STM32 采集 IMAGE, FS6818M4 端部署数字/字母 CNN 推理
进阶 8	多类型字符识别	已完成字母扩展	A-Z 字母独立固件域，另有中文 API/本机识别页面

1.3 设计目标

- (1) 完成 MCU 端从触摸轨迹到 28 x 28 图像再到神经网络推理的闭环。
- (2) 在同一块 STM32F103VE 上部署多个量化模型，形成速度、准确率、资源占用对比。
- (3) 建设网页上位机，使演示、数据采集、批量测试和固件烧录更加直观。
- (4) 扩展到字母识别和中文识别原型，展示多类型字符识别能力。
- (5) 与 FS6818M4 Linux 实验箱进行协同，完成语音播报和边缘推理接口设计。

2 总体方案设计

2.1 系统总体架构

系统采用“STM32 端采集与轻量推理 + PC 上位机管理 + Linux 实验箱协同”的三级架构。STM32 负责实时采集和低延迟识别；PC 上位机负责训练、导出、可视化、测试和固件管理；FS6818M4 Linux 实验箱用于跨平台语音输出以及更大模型边缘推理。

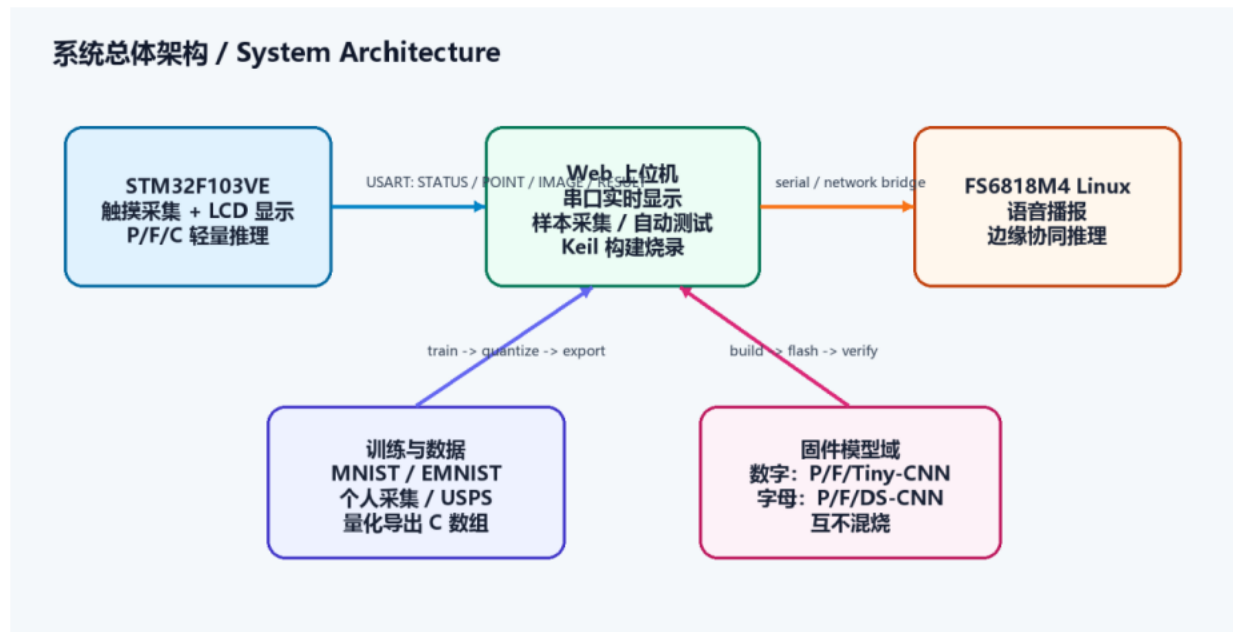


图 2-1 系统总体架构

2.2 数据流设计

一次识别从触摸点产生开始。触摸驱动周期性读取 XPT2046 坐标，滤除空触摸和异常跳点后在 LCD 上画线；按下 REC 后，预处理模块根据轨迹包围盒进行归一化，将笔画栅格化到 28 x 28 图像，并根据笔宽和加粗参数生成 uint8 灰度矩阵。识别器依次调用 Perceptron、FNN、Tiny-CNN/DS-CNN 三个模型，得到标签、置信度和耗时；结果一方面显示在 LCD 顶部，另一方面通过 USART1 发给上位机。

2.3 软硬件模块划分

层次	模块	主要职责
硬件层	STM32F103VE、LCD、触摸屏、USART、SWD、TF 卡	采集、显示、串口通信、下载调试和测试集载体
固件驱动层	lcd、uart、xpt2046、sdio/fatfs	提供外设初始化、绘图、串口发送和 TF 卡诊断接口
固件算法层	image_preprocess、perceptron、fnn、cnn、recognizer	完成预处理和定点推理
模型训练层	train_mnist.py、train_letters.py	训练 PyTorch 模型，导出量化权重和 C 文件
上位机层	web_dashboard_server.py、web 前端	串口交互、像素显示、样本采集、自动化测试、烧录管理
协同层	FS6818M4 Linux 脚本/服务	接收 STM32 串口帧，语音播报或边缘模型推理

3 硬件系统设计

3.1 主控与外设选择

主控选用 STM32F103VE，内核为 Cortex-M3，最高主频 72 MHz，片上 Flash 为 512 KB、SRAM 为 64 KB。该芯片资源足以容纳本项目的量化模型、触摸屏/LCD 驱动、串口协议和少量 TF 卡诊断代码。显示与触摸部分基于野火指南者开发板，LCD 为 ILI9341，触摸控制器为 XPT2046。

3.2 接口连接关系

接口/模块	连接方式	说明
LCD ILI9341	开发板板载 FSMC/SPI 相关接口	显示画板、按钮、识别结果和状态信息
XPT2046 触摸屏	开发板板载触摸接口	读取手写坐标，完成校准后映射到屏幕坐标
USART1	PA9/PA10, 115200 8N1	输出 STATUS、POINT、IMAGE、RESULT 等帧
SWD/CMSIS-DAP	PA13/PA14 + GND + 3V3 + 可选 NRST	Keil 下载与调试
TF 卡座	SDIO/FatFs 预留	组织测试集镜像，支持后续板端直接读卡
FS6818M4 实验箱	USB 转串口或网络桥接	接收推理结果或图像帧，完成语音播报/边缘推理

3.3 调试与下载方式

Keil 工程使用 CMSIS-DAP Debugger，Debug 与 Utilities 页面均选择 CMSIS-DAP，端口选择 Serial Wire。SWD 只负责下载调试，不负责串口回传；实时数据需要使用板载 USB 转串口或外接 CH340，网页上位机中选择对应 COM 口。

4 软件系统设计

4.1 固件目录与核心文件

实物工程位于 CourseDesign_DigitNN/keil_touch_digit_nn，基于野火“电阻触摸屏-触摸画板”例程改造。模型和算法核心位于 User/digit_nn/core 与 User/digit_nn/generated。

文件/目录	功能
main.c	系统初始化、触摸画板主循环、REC 触发识别
digit_nn/core/image_preprocess.c/h	轨迹采样、去噪、包围盒归一化、28 x 28 栅格化和加粗
digit_nn/core/perceptron.c/h	单层感知机定点推理
digit_nn/core/fnn.c/h	全连接网络定点推理
digit_nn/core/cnn.c/h	Tiny-CNN 与 DS-CNN 推理内核
digit_nn/core/recognizer.c/h	统一模型选择、标签和置信度输出
digit_nn/generated/*.c/*.h	当前激活域的量化模型权重
host_app/web_dashboard_server.py	本地网页服务、串口、测试、构建烧录和 API 接口

4.2 串口协议设计

串口协议采用文本帧，便于串口助手、网页和 Linux 实验箱同时解析。典型输出如下：

```
STATUS,state=idle,message=ready,proto=touch_stream_v1
POINT,x=120,y=88,pressure=1
STROKE,end=1
IMAGE,w=28,h=28,data=<1568 hex>
RESULT,model=P,label=2,label_index=2,confidence=30,time_us=...
```

```
RESULT,model=F,label=2,label_index=2,confidence=44,time_us=...
RESULT,model=C,label=2,label_index=2,confidence=45,time_us=...
```

其中 POINT 用于板端实时轨迹显示，IMAGE 是 REC 后的 28 x 28 模型输入，RESULT 是模型输出。Linux 实验箱只需要订阅 RESULT 即可完成语音播报；若执行边缘协同推理，则订阅 IMAGE 帧并在 Linux 端重新推理。

4.3 网页上位机设计

网页上位机以本地 Python 服务为后端，前端使用 HTML/CSS/JavaScript 实现。页面包括首页、数字工作区、字母工作区、自动化测试、中文识别和 TF 卡页。数字/字母工作区均支持“浏览器输入”和“单片机输入”切换，在单片机模式下显示板端实时轨迹；模型输入区以放大的 28 x 28 像素网格显示预处理结果，并同步展示 P/F/C 三个模型的标签、置信度和耗时。

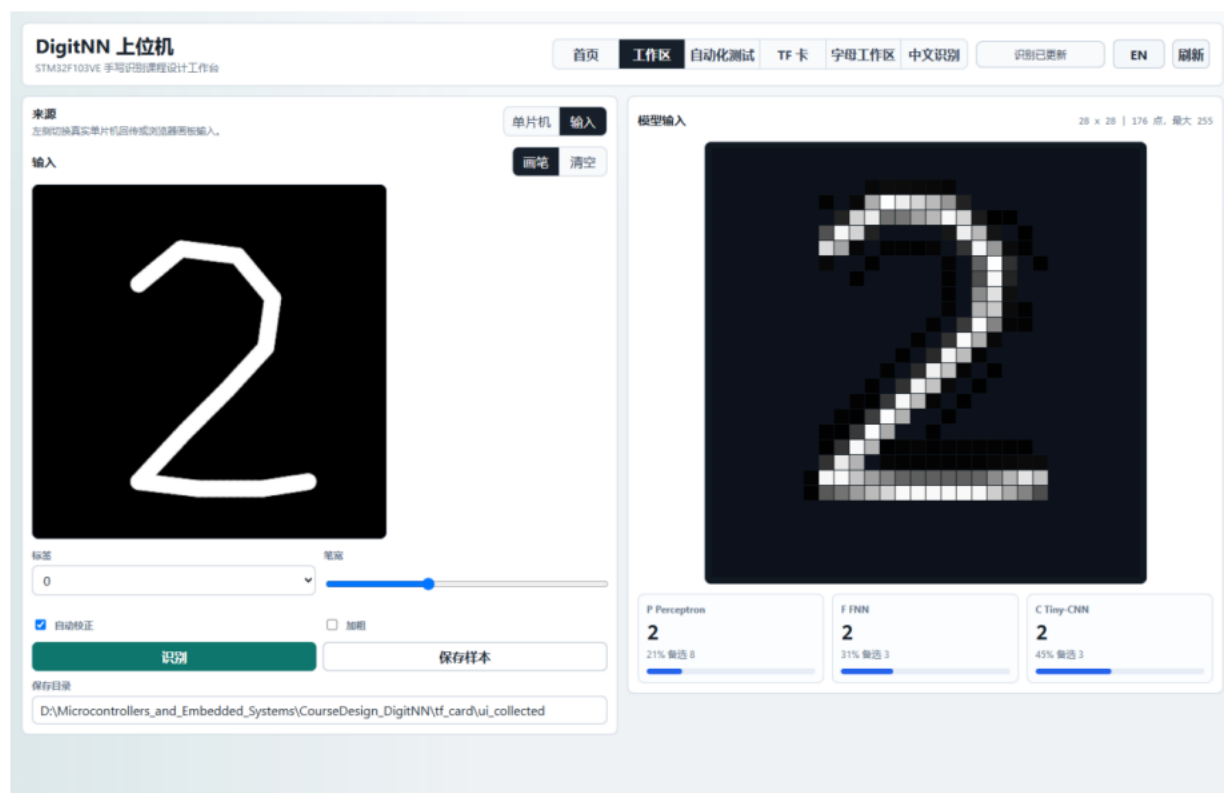


图 4-1 网页上位机数字工作区

4.4 构建与烧录流程

Web Dashboard 的 Firmware Deploy 面板封装了 Keil 命令行调用。Export 负责训练/导出权重，Build 负责编译，Flash 负责下载已有 AXF，Export+Flash 则串联导出、编译和下载。数字固件和字母固件分域导出，数字域不包含字母权重，字母域不包含数字权重，避免烧录时模型混用。

5 神经网络模型与量化部署

5.1 四类模型设计

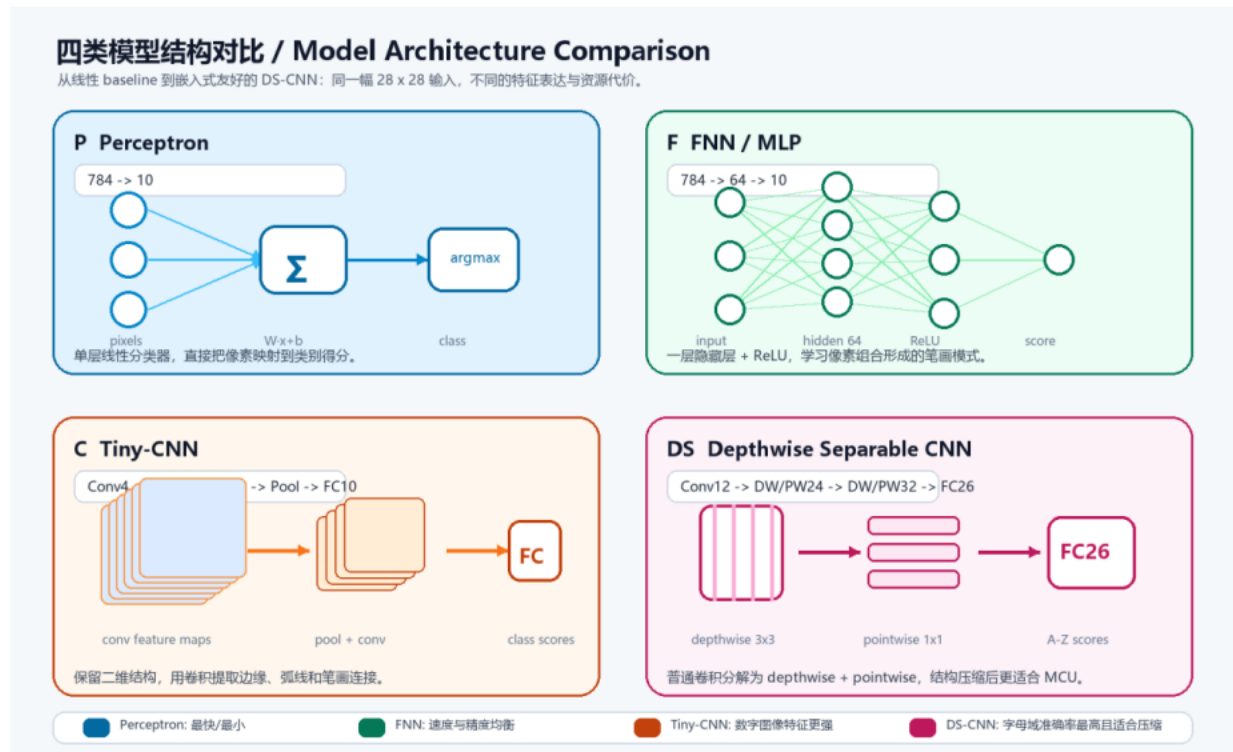


图 5-1 四类模型结构对比

感知机是最简单的线性分类器，将 28 x 28 图像展平成 784 维向量后直接输出各类别得分；FNN 在感知机基础上加入隐藏层和 ReLU，能够学习更复杂的像素组合模式；Tiny-CNN 保留二维图像结构，用卷积核提取边缘、弧线和笔画连接特征；DS-CNN 将普通卷积分解为 depthwise convolution 和 pointwise convolution，在保留卷积特征能力的同时显著减少参数和乘加次数。

表 5-1 模型结构与测试表现

模型	结构	训练轮数	代表测试准确率	说明
数字 Perceptron	784 -> 10	5	87.40% (MNIST 1000)	最小 baseline，完成基本任务
数字 FNN	784 -> 64 -> 10	5	92.90% (MNIST 1000)	全连接进阶模型，速度快
数字 Tiny-CNN	Conv4 -> Pool -> Conv8 -> Pool -> FC10	5	95.80% (MNIST 1000)	卷积模型，图像结构特征更强
Letter Perceptron	784 -> 26	8	66.40% (EMNIST Letters)	字母 baseline
Letter FNN	784 -> 96 -> 26	8	86.11% (EMNIST Letters)	字母全连接主力模型
Letter DS-CNN	Conv12 -> DW/PW -> DW/PW -> FC26	8	90.09% (EMNIST Letters)	字母域第三模型，替代普通 CNN

5.2 数字识别模型

数字域包含 Perceptron、FNN 和 Tiny-CNN 三个模型，类别数为 10，标签基准为字符 0。Perceptron 对应任务书基本要求；FNN 对应进阶全连接模型；Tiny-CNN 对应卷积模型训练与部署。三者共享同一幅 28 x 28 输入图像，使 LCD 和网页可直接比较 P/F/C 三种输出。

5.3 字母识别模型

字母域类别数为 26，标签基准为字符 A。项目训练了 Letter-Perceptron、Letter-FNN、Letter-Tiny-CNN 和 Letter-DS-CNN，最终以 Letter-DS-CNN 作为字母域 C 模型。DS-CNN 在完整 EMNIST Letters 测试集上达到 90.09%，略优于普通 Tiny-CNN 的 89.42%，因此更适合作为 MCU 端字母识别的第三模型。

5.4 int8 量化方法



图 5-2 int8 量化流程示意

量化不是简单地把小数截断为整数，而是为权重、激活和偏置分配尺度 scale。基本关系为 $r \approx \text{scale} \times (q - \text{zero_point})$ 。权重采用对称 int8 量化时 $\text{zero_point} = 0$ ，量化公式为 $q = \text{clip}(\text{round}(r / \text{scale}), -127, 127)$ 。推理时输入和权重相乘后使用 int32 累加器，偏置也按照 $\text{scale}_x \times \text{scale}_w$ 转为 int32，层间尺度再用 multiplier/shift 或右移近似完成。

本项目中输入图像使用 uint8_t 0..255，权重使用 int8_t，偏置和累加器使用 int32_t。这样可以将模型参数理论上压缩到 float32 的四分之一，并减少 STM32 上浮点乘法的开销。

5.5 中文识别与大字符集方案

对于约 5000 个常用汉字，STM32F103VE 不适合直接存储和推理完整中文模型。本项目将 STM32 定位为手写板，负责采集轨迹和生成图像；中文识别由上位机或视觉 API 处理。网页中

文识别页已支持 API 提示词、图像上传/板端图像输入和候选结果显示，为后续接入本机 OCR 模型或云端视觉模型预留接口。

6 关键算法与程序设计

6.1 手写轨迹预处理

预处理目标是把任意大小、任意位置的触摸轨迹转换为模型可接受的 28 x 28 灰度图。核心步骤包括：记录有效触摸点，按笔画连接相邻点；计算轨迹包围盒；保持宽高比例缩放到目标画布；对线段进行栅格化；根据笔宽参数做加粗；最后输出 784 个 uint8 灰度值。

```
for each touch point:
    append point to stroke buffer
    draw line on LCD
on REC:
    bbox = compute_bounding_box(points)
    normalize points into 28 x 28 canvas
    rasterize lines to uint8 image
    optional thicken and center-align
    run P / F / C recognizers
```

6.2 定点推理

以 FNN 为例，第一层将 784 个 uint8 输入与 int8 权重相乘并用 int32 累加，经过 ReLU 和右移后进入输出层。输出 logits 取最大值作为标签，最大值与次大值之间的差异换算为简化置信度。Perceptron 和 CNN/DS-CNN 采用相同的“int8 权重 + int32 累加”思想。

```
score = bias[class]
for i in range(784):
    score += int32(input[i]) * int32(weight[class][i])
label = argmax(score[0..class_count-1])
```

6.3 模型导出与固件同步

训练脚本在 PyTorch 中保存 .pt 浮点模型，同时生成 _quant.npz 和 C 头/源文件。--export-c 生成 firmware/generated，--export-keil 同步到 keil_touch_digit_nn/User/digit_nn/generated。RecognitionDomain.h 用于标识当前固件是 digit 域还是 letter 域。

6.4 代码规范落实

项目按照培训手册要求使用 C 语言蛇形命名、头文件/源文件分离、模块化目录和 Doxygen 风格注释。固件核心不使用动态内存分配，模型权重全部以 const 数组存放在 Flash 中；上位机脚本与训练脚本按工具职责拆分，避免把训练、串口和 UI 逻辑混在同一文件。

7 测试集制作与自动化测试

7.1 测试集组织

按照任务书“自备一张 TF 卡，制作至少两组测试集”的要求，项目建立了 tf_card 目录镜像，可复制到 TF 卡根目录。当前包含 MNIST 标准集、个人采集集、USPS 外部集和 EMNIST Letters 字母集。图片采用 BMP 格式，标签通过文件名和 label.txt/manifest.csv 记录。

```
tf_card/
├─ manifest.csv
```

```
| mnist/      # 1000 张 MNIST 标准数字 BMP
| personal/   # 129 张上位机/板端采集个人数字 BMP
| external_usps/ # 100 张 USPS 外部手写数字 BMP
| emnist_letters/ # 260 张 EMNIST Letters 字母 BMP
| ui_collected/ # 网页采集缓存
```

7.2 自动化测试方法

PC 端 `evaluate_tf_card.py` 对测试集中的 BMP 批量读取、预处理和推理，与真实标签比对后统计总数、正确数、准确率、平均推理时间和混淆类别。网页自动化测试页进一步把数字测试和字母测试拆成独立按钮，便于答辩时演示。

7.3 测试结果

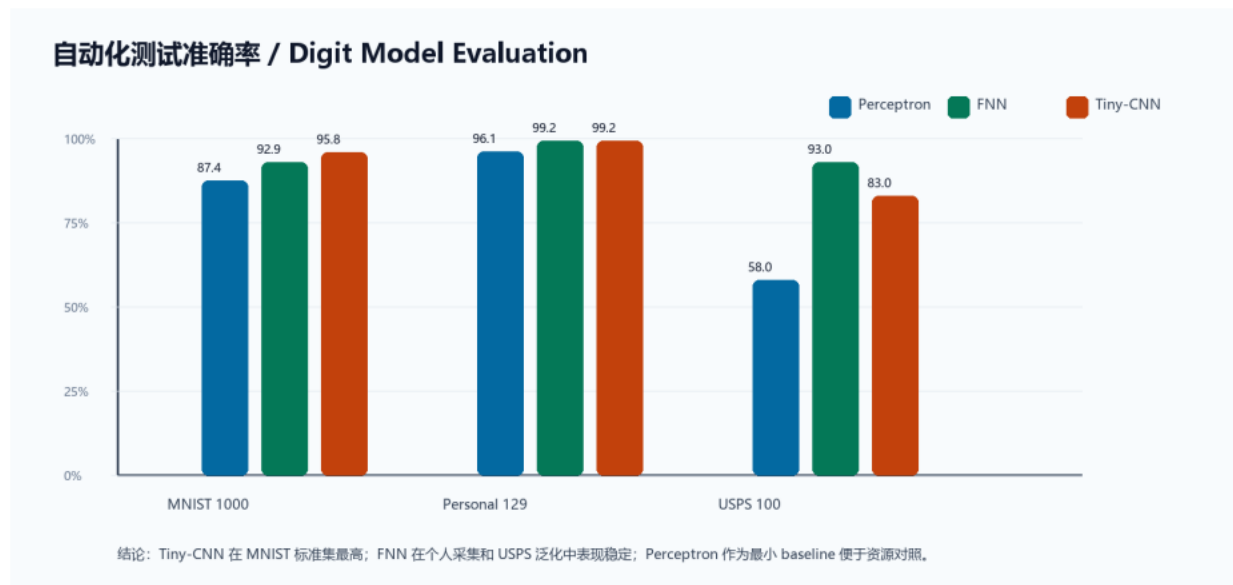


图 7-1 数字模型自动化测试准确率

表 7-1 数字模型批量测试结果

测试集	数量	模型	正确数	准确率	PC 平均推理时间
MNIST 标准集	1000	Perceptron	874	87.40%	688.6 us
MNIST 标准集	1000	FNN	929	92.90%	396.3 us
MNIST 标准集	1000	Tiny-CNN	958	95.80%	28117.6 us
个人采集集	129	Perceptron	124	96.12%	752.0 us
个人采集集	129	FNN	128	99.22%	593.3 us
个人采集集	129	Tiny-CNN	128	99.22%	29047.9 us
USPS 外部集	100	Perceptron	58	58.00%	800.9 us
USPS 外部集	100	FNN	93	93.00%	413.5 us
USPS 外部集	100	Tiny-CNN	83	83.00%	27921.5 us

MNIST 标准集上，Tiny-CNN 准确率最高；个人采集集上，FNN 与 Tiny-CNN 都达到 99.22%，说明上位机采集和预处理流程已经与模型输入分布较好匹配。USPS 外部集分布与 MNIST 差异较大，FNN 表现较稳定，Tiny-CNN 对个别 USPS 风格数字存在混淆，后续可继续增加数据增强或迁移测试优化。

7.4 易混淆类别分析

自动化测试会保存 confusions 字段。MNIST 标准集上，Perceptron 常见混淆包括 2 -> 8、4 -> 9、7 -> 9；FNN 常见混淆包括 4 -> 9、2 -> 7、8 -> 3；Tiny-CNN 错误数量更少，主要集中在 4 -> 9、2 -> 7、7 -> 9 等形状接近的类别。字母域中，I/L、G/Q、D/O、E/C 是高频混淆对，符合手写英文字母形态相似的特点。

8 系统联调与资源分析

8.1 Keil 编译与下载

Keil 工程目标为 DigitNN_Touch。调试过程中曾遇到 CMSIS-DAP 找不到设备、Flash Download failed、命令行烧录后未自动运行等问题，最终通过确认 DAP 连接、Debug/Utilities 同选 CMSIS-DAP、SWD 模式、串口与 SWD 分离、命令行 flash 后释放串口并复位运行等方式解决。

8.2 Flash/SRAM 占用

表 8-1 固件资源占用

固件域	Flash 占用	SRAM 占用	Flash 利用率	SRAM 利用率	说明
数字固件 (P/F/Tiny-CNN)	291388 B / 284.6 KB	18008 B / 17.6 KB	56.0%	27.5%	含网页当前三模型与 TF 卡诊断预留代码
字母固件 (P/F/DS-CNN)	370060 B / 361.4 KB	47928 B / 46.8 KB	70.6%	73.1%	含 A-Z 三模型，仍低于 STM32F103VE 资源上限

数字固件和字母固件均低于 STM32F103VE 资源上限。字母域由于类别数增至 26 且采用 DS-CNN，中间特征图和权重增加，SRAM 与 Flash 占用显著高于数字域。若去除 TF 卡/FatFs 诊断代码，固件体积还可进一步下降。

8.3 实物交互效果

实物板上 LCD 左侧为工具/按键区，右侧为手写画板。用户在画板区域写入数字或字母后点击 REC，顶部显示 P/F/C 三个模型的识别标签与置信度；网页上位机可同步显示板端轨迹、28 x 28 模型输入和串口日志。通过“浏览器/单片机”切换按钮，可比较浏览器画板输入和真实板端触摸输入的差异。

9 跨平台传输与边缘协同推理

9.1 跨平台传输：STM32 到 FS6818M4 语音播报

跨平台传输对应题目 6 进阶任务 (5) 和题目 7 的协作要求。STM32 端完成本地推理后，通过 USART1 输出 RESULT 帧；FS6818M4 Linux 实验箱通过 USB 转串口读取该帧，解析 label 和 confidence，然后调用本地 TTS、百度语音合成或系统播放器播报“识别结果为数字 2，置信度 45%”等语音。该方案使 STM32 保持轻量，Linux 端负责语音输出和更丰富的人机交互。

```
# Linux side pseudo code
line = serial.readline().decode()
```



```
if line.startswith('RESULT') and 'model=C' in line:
    label = parse_field(line, 'label')
    confidence = parse_field(line, 'confidence')
    speak(f'识别结果为 {label}, 置信度 {confidence}%')
```

9.2 边缘协同推理：STM32 采集，Linux 推理

边缘协同推理对应题目 6 进阶任务（7）。当任务需要更高精度或更大字符集时，STM32 只负责触摸采集和预处理，可把 IMAGE 帧发送到 FS6818M4；实验箱部署 PyTorch/ONNX/TFLite 模型，例如较大 CNN、MobileNetV3-small、ResNet-tiny 或 CRNN/Transformer OCR 模型，利用 Linux 侧更强算力完成数字、字母甚至中文识别。识别结果再回传上位机或由实验箱语音播报。

模式	STM32 职责	Linux 实验箱职责	适用场景
本地 MCU 推理	采集 + 预处理 + P/F/C 推理	可选语音播报	数字/字母轻量演示，低延迟
结果传输	采集 + MCU 推理 + 输出 RESULT	解析 RESULT 并语音播报	题目 6 与题目 7 跨平台展示
边缘协同推理	采集 + 输出 IMAGE 或 POINT	部署更大 CNN/DS-CNN/中文 OCR 推理	高准确率、多字符、大字符集

9.3 通信接口扩展

为了保证协同系统可扩展，建议 Linux 侧优先使用 IMAGE 帧而非屏幕截图。IMAGE 帧固定为 28 x 28，数据长度稳定，便于直接转为 NumPy tensor；若后续改用更高分辨率输入，可新增 IMAGE,w=56,h=56 或 STROKE_JSON 帧，并在协议版本中声明。

10 特色创新与不足

10.1 特色创新

- （1）同一 STM32 工程中实现 Perceptron、FNN、Tiny-CNN/DS-CNN 三模型对比，屏幕和网页同步显示置信度。
- （2）网页上位机不仅显示串口日志，还提供像素化模型输入、自动化测试、样本采集和 Keil 构建/烧录入口。
- （3）数字域和字母域完全分离导出，避免权重混烧，字母域引入 DS-CNN 替代普通 CNN。
- （4）首页可视化解释模型结构和量化机制，便于答辩时说明技术深度。
- （5）设计了 STM32 与 FS6818M4 的跨平台语音播报和边缘协同推理链路，为题目 6/7 合作提供接口。
- （6）中文识别采用“STM32 手写板 + PC/API 识别”的合理分工，规避 MCU 直接承载 5000 汉字模型的资源瓶颈。

10.2 不足与改进方向

- （1）数字 Tiny-CNN 在 PC Python 循环中推理时间较长，需进一步用 C/CMSIS-NN 或 Linux ONNX Runtime 做公平测速。

(2) 板端 TF 卡直接批量读取尚处于诊断预留阶段，当前自动化评估主要在 PC 本地完成。

(3) 数字 CNN 未达到任务书进阶项中“训练准确率 99% 以上”的理想目标，后续可在 Linux 实验箱部署更大 CNN 完成该目标。

(4) 字母 IL、G/Q、D/O 混淆仍较明显，可通过更多个性化采集、旋转平移增强或 QAT 改进。

(5) 中文识别目前以 API/本机模型接口为主，尚未训练本地 5000 字专用模型。

11 总结

本课程设计完成了基于 STM32F103VE 的手写数字/字母识别系统。从触摸屏采集、图像预处理、神经网络训练、量化导出、Keil 部署，到网页上位机和自动化测试，形成了完整的嵌入式 AI 开发闭环。基本任务中的触摸采集、感知机训练、权重导出和 MCU 推理均已完成；进阶部分实现了 FNN、CNN/DS-CNN、测试集、自动化测试、多类型字符、跨平台语音播报和边缘协同推理方案。

实验表明，在资源受限的 STM32F103VE 上，int8 量化能显著降低模型存储和计算压力，使多个轻量神经网络可以同时运行；同时，PC/Web/Linux 协同能够弥补 MCU 在可视化、数据管理、大模型推理和语音交互方面的不足。通过本设计，较完整地掌握了微控制器外设驱动、串口协议、神经网络模型训练、量化部署、上位机开发和跨平台协同的综合实践方法。

参考文献

- [1] 《微控制器与嵌入式系统》课程设计任务书，自动化学院，2026。
- [2] 《微控制器与嵌入式系统》课程设计培训资料，自动化学院，2026。
- [3] 野火 STM32 库开发实战指南--基于野火指南者开发板。
- [4] LeCun Y, Cortes C, Burges C. The MNIST Database of Handwritten Digits.
- [5] Cohen G, Afshar S, Tapson J, van Schaik A. EMNIST: Extending MNIST to handwritten letters.
- [6] PyTorch 官方文档：模型训练、张量运算与模型保存。
- [7] STMicroelectronics. STM32F103xC/D/E Reference Manual.

附录 A 主要文件清单

类别	路径
Keil 工程	CourseDesign_DigitNN/keil_touch_digit_nn/Project/RVMDK (uv5) /BH-F103.uvprojx
固件核心算法	CourseDesign_DigitNN/keil_touch_digit_nn/User/digit_nn/core/
当前模型权重	CourseDesign_DigitNN/keil_touch_digit_nn/User/digit_nn/generated/
训练脚本	CourseDesign_DigitNN/tools/train_mnist.py, tools/train_letters.py
测试脚本	CourseDesign_DigitNN/tools/evaluate_tf_card.py
网页上位机	CourseDesign_DigitNN/host_app/web_dashboard_server.py, host_app/web/
测试集镜像	CourseDesign_DigitNN/tf_card/
字母模型包	CourseDesign_DigitNN/packages/letter_models_20260706.zip

附录 B 常用运行命令

```
cd CourseDesign_DigitNN
python tools\train_mnist.py --model perceptron --epochs 5
python tools\train_mnist.py --model fnn --epochs 5 --augment
python tools\train_mnist.py --model cnn --epochs 5 --augment
python tools\train_letters.py --model all --epochs 8 --augment
python tools\evaluate_tf_card.py
python host_app\web_dashboard_server.py
python tools\keil_flash.py --action flash --uv4 D:\UV4\UV4.exe
```